

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Firewall Traffic Control and Forensic Verification

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 18/09/2026

EXECUTIVE SUMMARY

This practical exercise examined the use of a Linux host-based firewall to control HTTP traffic and the forensic verification of firewall behaviour. The exercise was conducted within an isolated laboratory environment using Kali Linux, Apache2, iptables, curl and TShark/Wireshark.

The practical began with preservation of the existing firewall configuration. The active iptables ruleset was exported, recorded with packet and byte counters, and hashed using SHA-256 to provide an integrity reference for the original firewall state. The network configuration and Apache HTTP listener were then verified to establish the baseline condition of the host before firewall modification.

The laboratory HTTP resource, `firewall_lab.html`, was hosted by Apache2 and made available through TCP port 80. The server's primary laboratory interface was identified as `eth0`, with IPv4 address `192.168.232.128/24`. The HTTP service was verified to be listening before firewall testing commenced.

A successful baseline HTTP request was subsequently generated against the laboratory web resource. The verbose curl output demonstrated successful TCP connectivity and an HTTP 200 OK response from Apache2. This baseline provided application-level evidence of normal HTTP service availability before any firewall restriction was introduced.

The practical was designed to proceed from this baseline to packet capture, implementation of a narrowly scoped source-specific iptables DROP rule, observation of

blocked HTTP behaviour, correlation of packet evidence with firewall counters, and eventual removal of the rule to confirm restoration. The evidence collected during each stage was intended to demonstrate the relationship between firewall configuration, network packets, TCP behaviour and application-level connectivity.

PURPOSE OF THE ASSIGNMENT

The purpose of this practical was to investigate and demonstrate how a Linux host-based firewall can control HTTP traffic and how firewall activity can be verified using network-forensic evidence.

The exercise was specifically designed to establish a normal HTTP baseline, preserve the original firewall configuration, introduce a narrowly scoped firewall restriction, observe the resulting network behaviour, and correlate the observed traffic with firewall rule counters.

From a digital-forensics perspective, the exercise also demonstrated the importance of preserving the original system state, documenting changes, maintaining evidence integrity through hashing, and distinguishing between application, network and firewall evidence when determining the cause of a connectivity failure.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the practical were to:

1. Explain the role of host-based and network-based firewalls in controlling network traffic.
2. Identify and interpret the purpose of the INPUT, OUTPUT and FORWARD iptables chains.
3. Preserve the original firewall configuration before making any modification.
4. Establish and document the normal network and HTTP service state before firewall intervention.
5. Generate and capture HTTP traffic for forensic examination.
6. Implement a narrowly scoped source-specific TCP/80 DROP rule using iptables.
7. Verify the presence and operation of the firewall rule using rule listings and packet/byte counters.
8. Compare allowed and blocked HTTP behaviour using curl and packet-capture evidence.

9. Identify TCP retransmissions, incomplete handshakes and timeout behaviour associated with silently dropped traffic.
10. Explain the forensic difference between DROP and REJECT behaviour.
11. Remove the firewall rule safely and verify restoration of HTTP connectivity.
12. Preserve relevant evidence and calculate SHA-256 hashes for integrity verification.

METHODOLOGY OF THE ASSIGNMENT

The practical followed a controlled forensic workflow designed to establish a baseline before modifying the firewall.

First, a dedicated laboratory workspace was created under /home/kali/SBT-DF203-Lab6. Separate directories were created for evidence, working files, exported artifacts, reports, screenshots and scripts. This provided a structured location for preserving original evidence and separating it from analysis outputs.

Second, the existing iptables configuration was preserved before any firewall modification. The ruleset was exported using iptables-save, while a verbose rule listing containing packet and byte counters was also recorded. A SHA-256 hash was calculated for the exported ruleset to provide an integrity reference.

Third, the host's network configuration and routing information were recorded. The active laboratory interface, IPv4 address and routing table were examined. Apache2 was then verified using ss to establish that TCP port 80 was actively listening before firewall testing. Fourth, baseline HTTP connectivity was tested using curl against the laboratory Apache resource:

http://192.168.232.128/firewall_lab.html

The verbose curl output was used to establish the normal application-level behaviour of the HTTP service, including successful TCP connection establishment and the HTTP response returned by Apache.

The subsequent forensic workflow was designed to capture the allowed HTTP session, inspect the TCP three-way handshake and HTTP request/response, preserve the resulting PCAPNG evidence and calculate its SHA-256 hash.

After preservation of the allowed baseline, the practical would proceed to the controlled firewall modification. A source-specific TCP/80 DROP rule would be inserted into the

INPUT chain for the authorised laboratory client. The rule would then be verified and its packet/byte counters monitored during a blocked HTTP attempt.

The blocked traffic would be examined for evidence of an incomplete TCP connection, retransmitted SYN packets and the absence of a normal server response. These observations would then be correlated with the iptables packet and byte counters.

Finally, the exact firewall rule would be removed. The restored ruleset and HTTP service would be verified to confirm that the temporary firewall modification had been successfully reversed.

TOOLS AND RESOURCES USED

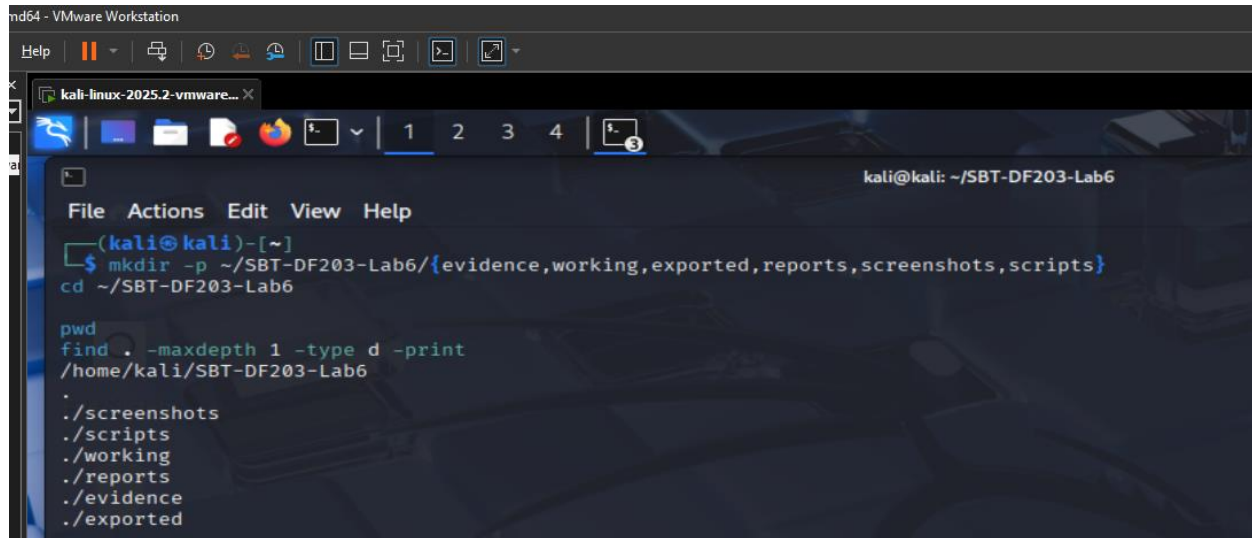
The following tools and resources were used during the practical:

Tool/Resource	Purpose
Kali Linux	Laboratory operating system and forensic workstation
Apache2	HTTP server used to host the laboratory webpage
iptables	Host-based firewall configuration and traffic control
iptables-save	Preservation/export of the firewall ruleset
ip	Network interface and routing verification
ss	Verification of the Apache TCP/80 listener
curl	Generation and verification of HTTP requests
TShark	Command-line packet capture and packet analysis
Wireshark	Graphical packet inspection and forensic analysis
SHA-256	Integrity verification of exported evidence
Terminal	Execution and recording of laboratory commands
Laboratory webpage	Controlled HTTP resource used for testing

Lab Workspace and Evidence Directory

The practical exercise was initiated by creating a dedicated working directory for the firewall traffic-control investigation. Separate directories were created for evidence, working files, exported artifacts, reports, screenshots, and scripts. This structure was established to maintain an organized evidence trail throughout the exercise and to

distinguish original packet captures and configuration evidence from generated analysis outputs. The working directory was confirmed as /home/kali/SBT-DF203-Lab6, with all required subdirectories successfully created.



```
md64 - VMware Workstation
kali-linux-2025.2-vmware... X
File Actions Edit View Help
(kali@kali)~[~]
$ mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab6

pwd
/home/kali/SBT-DF203-Lab6
find . -maxdepth 1 -type d -print
./screenshots
./scripts
./working
./reports
./evidence
./exported
```

Preservation of the Original Firewall Configuration

Before making any changes to the host firewall, the existing iptables configuration was exported and preserved as the baseline ruleset. The active rules were also recorded with packet and byte counters to provide an initial reference point for later comparison. A SHA-256 hash was generated for the exported ruleset to provide an integrity reference for the original firewall configuration. This step ensured that subsequent firewall modifications could be compared against the original state and that the baseline configuration could be verified during restoration.

```
md64 - VMware Workstation
Help
kali-linux-2025.2-vmware... X
kali@kali: ~/SBT-DF203-Lab6
File Actions Edit View Help
(kali@kali)~$ sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
[sudo] password for kali:
# Generated by iptables-save v1.8.11 (nf_tables) on Fri Sep 18 07:38:02 2026
*raw
:PREROUTING ACCEPT [3996:546851]
:OUTPUT ACCEPT [0:0]
-A PREROUTING -d 172.18.0.2/32 ! -i br-d2696585dc9a -j DROP
-A PREROUTING -d 172.18.0.3/32 ! -i br-d2696585dc9a -j DROP
COMMIT
# Completed on Fri Sep 18 07:38:02 2026
# Generated by iptables-save v1.8.11 (nf_tables) on Fri Sep 18 07:38:02 2026
*filter
:INPUT DROP [12:7620]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [43:1720]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-INTERNAL - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-USER - [0:0]
:ufw-after-forward - [0:0]
:ufw-after-input - [0:0]
:ufw-after-logging-forward - [0:0]
:ufw-after-logging-input - [0:0]
:ufw-after-logging-output - [0:0]
:ufw-after-output - [0:0]
:ufw-before-forward - [0:0]
:ufw-before-input - [0:0]
:ufw-before-logging-forward - [0:0]
:ufw-before-logging-input - [0:0]
:ufw-before-logging-output - [0:0]
:ufw-before-output - [0:0]
:ufw-logging-allow - [0:0]
:ufw-logging-deny - [0:0]
:ufw-not-local - [0:0]
:ufw-reject-forward - [0:0]
:ufw-reject-input - [0:0]
:ufw-reject-output - [0:0]
```

```
kali-linux-2025.2-vmware... X
kali@kali: ~/SBT-DF203-Lab6

File Actions Edit View Help
-A INPUT -j ufw-before-logging-input
-A INPUT -j ufw-before-input
-A INPUT -j ufw-after-input
-A INPUT -j ufw-after-logging-input
-A INPUT -j ufw-reject-input
-A INPUT -j ufw-track-input
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A FORWARD -j ufw-before-logging-forward
-A FORWARD -j ufw-before-forward
-A FORWARD -j ufw-after-forward
-A FORWARD -j ufw-after-logging-forward
-A FORWARD -j ufw-reject-forward
-A FORWARD -j ufw-track-forward
-A OUTPUT -j ufw-before-logging-output
-A OUTPUT -j ufw-before-output
-A OUTPUT -j ufw-after-output
-A OUTPUT -j ufw-after-logging-output
-A OUTPUT -j ufw-reject-output
-A OUTPUT -j ufw-track-output
-A DOCKER -d 172.18.0.3/32 ! -i br-d2696585dc9a -o br-d2696585dc9a -p tcp -m tcp --dport 8000 -j ACCEPT
-A DOCKER -d 172.18.0.2/32 ! -i br-d2696585dc9a -o br-d2696585dc9a -p tcp -m tcp --dport 3000 -j ACCEPT
-A DOCKER ! -i br-d2696585dc9a -o br-d2696585dc9a -j DROP
-A DOCKER ! -i br-ec90d8c3abfb -o br-ec90d8c3abfb -j DROP
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o br-d2696585dc9a -j DOCKER
-A DOCKER-BRIDGE -o br-ec90d8c3abfb -j DOCKER
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o br-d2696585dc9a -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-CT -o br-ec90d8c3abfb -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-INTERNAL
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i br-d2696585dc9a -j ACCEPT
-A DOCKER-FORWARD -i br-ec90d8c3abfb -j ACCEPT
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-USER -j RETURN
-A ufw-after-input -p udp -m udp --dport 137 -j ufw-skip-to-policy-input
-A ufw-after-input -p udp -m udp --dport 138 -j ufw-skip-to-policy-input
-A ufw-after-input -p tcp -m tcp --dport 139 -j ufw-skip-to-policy-input
-A ufw-after-input -p tcp -m tcp --dport 445 -j ufw-skip-to-policy-input
-A ufw-after-input -p udp -m udp --dport 67 -j ufw-skip-to-policy-input
```

```
kali@kali: ~/SBT-DF203-Lab6

File Actions Edit View Help

*nat
:PREROUTING ACCEPT [222:19100]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [4825:289538]
:POSTROUTING ACCEPT [4825:289538]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A POSTROUTING -s 172.19.0.0/16 ! -o br-ec90d8c3abfb -j MASQUERADE
-A POSTROUTING -s 172.18.0.0/16 ! -o br-d2696585dc9a -j MASQUERADE
-A DOCKER ! -i br-d2696585dc9a -p tcp -m tcp --dport 3000 -j DNAT --to-destination 172.18.0.2:3000
-A DOCKER ! -i br-d2696585dc9a -p tcp -m tcp --dport 8000 -j DNAT --to-destination 172.18.0.3:8000
COMMIT
# Completed on Fri Sep 18 07:38:02 2026
Chain INPUT (policy DROP 12 packets, 7620 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1     4100 580K ufw-before-logging-input all -- *      *       0.0.0.0/0             0.0.0.0/0
2     4100 580K ufw-before-input all -- *      *       0.0.0.0/0             0.0.0.0/0
3      219 25880 ufw-after-input all -- *      *       0.0.0.0/0             0.0.0.0/0
4        12 7620 ufw-after-logging-input all -- *      *       0.0.0.0/0             0.0.0.0/0
5        12 7620 ufw-reject-input all -- *      *       0.0.0.0/0             0.0.0.0/0
6        12 7620 ufw-track-input all -- *      *       0.0.0.0/0             0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER-USER all -- *      *       0.0.0.0/0             0.0.0.0/0
2      0      0 DOCKER-FORWARD all -- *      *       0.0.0.0/0             0.0.0.0/0
3      0      0 ufw-before-logging-forward all -- *      *       0.0.0.0/0             0.0.0.0/0
4      0      0 ufw-before-forward all -- *      *       0.0.0.0/0             0.0.0.0/0
5      0      0 ufw-after-forward all -- *      *       0.0.0.0/0             0.0.0.0/0
6      0      0 ufw-after-logging-forward all -- *      *       0.0.0.0/0             0.0.0.0/0
7      0      0 ufw-reject-forward all -- *      *       0.0.0.0/0             0.0.0.0/0
8      0      0 ufw-track-forward all -- *      *       0.0.0.0/0             0.0.0.0/0

Chain OUTPUT (policy ACCEPT 43 packets, 1720 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1    5125 327K ufw-before-logging-output all -- *      *       0.0.0.0/0             0.0.0.0/0
2    5125 327K ufw-before-output all -- *      *       0.0.0.0/0             0.0.0.0/0
3    4865 291K ufw-after-output all -- *      *       0.0.0.0/0             0.0.0.0/0
4    4865 291K ufw-after-logging-output all -- *      *       0.0.0.0/0             0.0.0.0/0
5    4865 291K ufw-reject-output all -- *      *       0.0.0.0/0             0.0.0.0/0
6    4865 291K ufw-track-output all -- *      *       0.0.0.0/0             0.0.0.0/0
```

```
kali@kali: ~/SBT-DF203-Lab6

File Actions Edit View Help

num  pkts bytes target    prot opt in     out     source                 destination
Chain ufw-track-input (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
Chain ufw-track-output (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1     10     600 ACCEPT    tcp -- *      *       0.0.0.0/0             0.0.0.0/0      ctstate NEW
2    4812 289K ACCEPT    udp -- *      *       0.0.0.0/0             0.0.0.0/0      ctstate NEW
Chain ufw-user-forward (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
Chain ufw-user-input (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 ACCEPT    tcp -- *      *       0.0.0.0/0             0.0.0.0/0      tcp dpt:22
2      0      0 ACCEPT    tcp -- *      *       0.0.0.0/0             0.0.0.0/0      tcp dpt:9091
Chain ufw-user-limit (0 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 LOG       all -- *      *       0.0.0.0/0             0.0.0.0/0      limit: avg 3/min burst 5 LOG flags 0 level 4 prefix "[UFW LI
MIT BLOCK]"
2      0      0 REJECT    all -- *      *       0.0.0.0/0             0.0.0.0/0      reject-with icmp-port-unreachable
Chain ufw-user-limit-accept (0 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 ACCEPT    all -- *      *       0.0.0.0/0             0.0.0.0/0
Chain ufw-user-logging-forward (0 references)
num  pkts bytes target    prot opt in     out     source                 destination
Chain ufw-user-logging-input (0 references)
num  pkts bytes target    prot opt in     out     source                 destination
Chain ufw-user-logging-output (0 references)
num  pkts bytes target    prot opt in     out     source                 destination
Chain ufw-user-output (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
72623b5d5b59a58f57db29b9df34a514f14cc97f8b51bdb4d5e52597724ab290 reports/iptables_before.rules
```

Lab Network Configuration and Apache Listener

Before conducting any firewall modification or traffic-capture activity, the network configuration of the protected server was recorded to establish the baseline environment

and identify the interface and IP address that would be used throughout the investigation. The `ip -br address` command showed that the primary active network interface was `eth0`, with the IPv4 address `192.168.232.128/24`. The interface was in an UP state, confirming that the server had an active network connection. Other interfaces, including Docker bridge and virtual Ethernet interfaces, were also present on the system but were not used for the client-to-server HTTP test.

The server routing table was then examined using `ip route`. The output confirmed that the `192.168.232.0/24` network was directly connected through `eth0`, with `192.168.232.2` configured as the default gateway. This established the expected network path for communication between the laboratory client and the protected server and provided useful network context for interpreting the subsequent packet captures.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
lo                UNKNOWN    127.0.0.1/8 ::1/128
eth0              UP        192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0          DOWN     172.17.0.1/16
br-d2696585dc9a  UP       172.18.0.1/16 fe80::48e2:5cff:fe88:d9d0/64
br-ec90d8c3abfb  DOWN     172.19.0.1/16
veth521afec@if2  UP       fe80::b022:6dff:fe77:bd4/64
veth0caa175@if2  UP       fe80::b483:feff:fe51:5e43/64
```

The availability of the HTTP service was also verified before any firewall rule was introduced. The `ss -ltnp` command confirmed that the Apache2 web server was actively listening on TCP port 80 on all IPv4 interfaces, as indicated by the `*:80` listener associated with the `apache2` processes. This verification was important from a forensic perspective because it established that the web service was available at the beginning of the experiment. Therefore, if HTTP access subsequently failed after the source-specific firewall rule was introduced, the failure could be investigated in relation to the firewall rule rather than being immediately attributed to an unavailable or stopped Apache service. The additional listeners on port 8000 were associated with Docker and were outside the scope of this HTTP firewall experiment.

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP          192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0          DOWN        172.17.0.1/16
br-d2696585dc9a  UP          172.18.0.1/16 fe80::48e2:5cff:fe88:d9d0/64
br-ec90d8c3abfb  DOWN        172.19.0.1/16
veth521afec@if2  UP          fe80::b022:6dff:fe77:bd4/64
veth0caa175@if2  UP          fe80::b483:feff:fe51:5e43/64

(kali@kali)-[~/SBT-DF203-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100

```

The server baseline was therefore established as 192.168.232.128/24 on eth0, with a directly connected 192.168.232.0/24 network and Apache2 listening on TCP port 80. These observations were recorded before modifying the firewall and were used as reference evidence for the subsequent allowed-traffic capture, blocked-traffic investigation, firewall-counter correlation, and restoration testing.

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      4096      0.0.0.0:*      users:((("docker-proxy",pid=370682,fd=8))
LISTEN 0      511      *:80          *:*(("apache2",pid=345882,fd=4),("apache2",pid=345881,fd=4),("apache2",pid=345880,fd=4),("ap
ache2",pid=345876,fd=4),("apache2",pid=345875,fd=4),("apache2",pid=1326,fd=4))
LISTEN 0      4096      [::]:8000     [::]:*(("docker-proxy",pid=370689,fd=8))

```

Baseline HTTP Connectivity

Following verification of the server's network configuration and Apache listener, the next step is to establish whether the authorised laboratory client can successfully access the HTTP service **before the firewall blocking rule is introduced**. This baseline test is important because it establishes the normal behaviour of the client-server connection against which the later blocked condition can be compared.

From the client VM, the client IP address should first be recorded using `ip -br address`.

The client should then request the training webpage hosted on the server using `curl`:

```
curl -v --connect-timeout 5 http://192.168.232.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
```

```
kali@kali: ~/SBT-DF203-Lab6
File Actions Edit View Help
(kali@kali)~$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      4096      0.0.0.0:8000      0.0.0.0:*      users:((("docker-proxy",pid=370682,fd=8))
LISTEN 0      511      *:80              *:~          users:((("apache2",pid=345882,fd=4),("apache2",pid=345881,fd=4),("ap
apache2",pid=345876,fd=4),("apache2",pid=345875,fd=4),("apache2",pid=1326,fd=4))
LISTEN 0      4096      [::]:8000        [::]:*      users:((("docker-proxy",pid=370689,fd=8))

(kali@kali)~$ curl -v --connect-timeout 5 http://192.168.232.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.232.128:80 ...
* Established connection to 192.168.232.128 (192.168.232.128 port 80) from 192.168.232.128 port 55142
% Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
0         0     0    0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.232.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404 Not Found
< Date: Fri, 18 Sep 2026 11:51:10 GMT
< Server: Apache/2.4.68 (Debian)
< Content-Length: 317
< Content-Type: text/html; charset=iso-8859-1
<
{ [317 bytes data]
100    317 100    317    0    0 61721    0      0      0      0      0
* Connection #0 to host 192.168.232.128:80 left intact
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at 192.168.232.128 Port 80</address>
</body></html>

(kali@kali)~$
```

A successful result should demonstrate that the client can establish a TCP connection to 192.168.232.128 on port 80 and receive the Apache-hosted firewall_lab.html page. The verbose curl output provides application-level evidence of the connection, including the destination address, TCP connection status, HTTP request and HTTP response. This baseline will subsequently be compared with the behaviour observed after the source-specific DROP rule is applied.

Server Network Interface Verification

The server network configuration was first verified to establish the IP address and active interface that would be used throughout the firewall traffic-control exercise. The ip -br address command showed that the primary eth0 interface was active and assigned the IPv4 address 192.168.232.128/24. Additional Docker bridge and virtual Ethernet interfaces were also present on the host but were associated with container networking. The eth0 interface and its assigned IPv4 address were therefore identified as the relevant server network details for the subsequent HTTP connectivity and packet-capture tests.

```
(kali@kali)~$ ip -br address | tee reports/server_interfaces.txt
lo                UNKNOWN      127.0.0.1/8  ::1/128
eth0              UP          192.168.232.128/24  fe80::d5cc:fb25:aa2d:b901/64
docker0          DOWN        172.17.0.1/16
br-d2696585dc9a  UP          172.18.0.1/16  fe80::48e2:5cff:fe88:d9d0/64
br-ec90d8c3abfb  DOWN        172.19.0.1/16
veth521afec@if2  UP          fe80::b022:6dff:fe77:bd4/64
veth4a037e0@if2  UP          fe80::60bc:2aff:fed8:7eab/64
```

Server Routing Verification

The server routing table was subsequently examined to confirm the available network paths before conducting the HTTP tests. The output showed that the 192.168.232.0/24 network was directly connected through eth0, with the server using 192.168.232.128 as its source address on that network. A default route through 192.168.232.2 was also present. Recording the routing configuration established the network path available to the server before any firewall rule was introduced.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
```

Apache HTTP Service Verification

Before testing firewall behaviour, the availability of the HTTP service on the server was verified using the ss utility. The output confirmed that the Apache web server was in a LISTEN state on TCP port 80 and was bound to *:80, indicating that the service was actively accepting HTTP connections on the server's IPv4 interfaces. This verification established that the HTTP service was operational before the firewall rule was introduced and provides an important baseline for distinguishing a firewall-induced connection failure from an unavailable web service.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      511          *:80          *:*    users:((("apache2",pid=345882,fd=4),("apache2",pid=345881,fd=4),("apache2",pid=345880,fd=4),("ap
ache2",pid=345876,fd=4),("apache2",pid=345875,fd=4),("apache2",pid=1326,fd=4)))
(kali@kali)-[~/SBT-DF203-Lab6]
```

Server network configuration

Before conducting the firewall traffic-control tests, the server's network configuration was verified to establish the active interface and IPv4 address that would be used during the exercise. The ip -br address output showed that the primary eth0 interface was active and assigned the IPv4 address 192.168.232.128/24. Additional virtual interfaces associated with Docker networking were also present but were not used as the primary interface for the HTTP firewall test. This information established the server address and interface required for subsequent connectivity testing and packet capture.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0 DOWN 172.17.0.1/16
br-d2696585dc9a UP 172.18.0.1/16 fe80::48e2:5cff:fe88:d9d0/64
br-ec90d8c3abfb DOWN 172.19.0.1/16
veth521afec@if2 UP fe80::b022:6dff:fe77:bd4/64
veth4a037e0@if2 UP fe80::60bc:2aff:fed8:7eab/64
```

Server routing configuration

The server's routing table was examined to confirm the available network path before introducing any firewall changes. The output showed that the 192.168.232.0/24 network was directly connected through eth0, while a default route was configured through 192.168.232.2. Recording the routing information provided a baseline for the server's network connectivity and helped establish that the HTTP traffic would use the expected network interface during the firewall test.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
```

Apache HTTP service verification

Before testing the firewall behaviour, the HTTP service on the server was verified using the ss utility. The output confirmed that Apache was in a LISTEN state on TCP port 80 and was bound to *:80. This established that the web service was operational and accepting HTTP connections before the firewall restriction was introduced. The verification was important because it provides a service-level baseline for distinguishing a firewall-induced connection failure from a web-server availability problem.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0 511 *:80 *:80 users:((("apache2",pid=345882,fd=4),("apache2",pid=345881,fd=4),("apache2",pid=345880,fd=4),("ap
ache2",pid=345876,fd=4),("apache2",pid=345875,fd=4),("apache2",pid=1326,fd=4)))

(kali@kali)-[~/SBT-DF203-Lab6]
$ ip -br address
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0 DOWN 172.17.0.1/16
br-d2696585dc9a UP 172.18.0.1/16 fe80::48e2:5cff:fe88:d9d0/64
br-ec90d8c3abfb DOWN 172.19.0.1/16
veth521afec@if2 UP fe80::b022:6dff:fe77:bd4/64
veth0a88221@if2 UP fe80::3027:61ff:fe3b:8e44/64
```

Successful HTTP baseline

A baseline HTTP connectivity test was performed before any source-specific firewall restriction was introduced. The request to http://192.168.232.128/firewall_lab.html successfully established a TCP connection to the server on port 80, and Apache returned an HTTP/1.1 200 OK response.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo tee /var/www/html/firewall_lab.html > /dev/null <<'EOF'
<!DOCTYPE html>
<html>
<head>
  <title>Firewall Lab</title>
</head>
<body>
  <h1>Firewall Traffic Control Lab</h1>
  <p>HTTP connectivity is working before the firewall restriction.</p>
</body>
</html>
EOF
[sudo] password for kali:
```

The response also identified the Apache server and returned the expected laboratory webpage content. This confirmed that the HTTP service, TCP connectivity, and requested web resource were functioning normally before the firewall rule was applied.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://192.168.232.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.232.128:80 ...
* Established connection to 192.168.232.128 (192.168.232.128 port 80) from 192.168.232.128 port 47310
% Total % Received % Xferd Average Speed Time Time Time Current
 0     0    0     0      0      0      0      0 0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.232.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 16:36:59 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 16:36:24 GMT
< ETag: "d0-65bc48047211a"
< Accept-Ranges: bytes
< Content-Length: 208
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [208 bytes data]
100 208 100 208 0 16736 0
* Connection #0 to host 192.168.232.128:80 left intact
<!DOCTYPE html>
<html>
<head>
  <title>Firewall Lab</title>
</head>
<body>
  <h1>Firewall Traffic Control Lab</h1>
  <p>HTTP connectivity is working before the firewall restriction.</p>
</body>
</html>
```

CONCLUSION

The practical established a controlled baseline for investigating Linux host-based firewall behaviour and HTTP traffic. The laboratory workspace and evidence directories were created before testing, and the original firewall configuration was preserved and hashed before any modification.

The network configuration identified eth0 with IPv4 address 192.168.232.128/24 as the primary laboratory interface. The routing table confirmed connectivity through the 192.168.232.0/24 network, while Apache2 was verified to be listening on TCP port 80.

A baseline HTTP request to http://192.168.232.128/firewall_lab.html successfully reached Apache2 and returned an HTTP 200 OK response. This demonstrated that the HTTP service was operational before firewall intervention and established the reference condition required for subsequent forensic comparison.

The remaining stages of the practical are intended to build on this baseline by preserving the allowed HTTP packet capture, introducing a narrowly scoped source-specific DROP rule, observing the resulting blocked connection behaviour, correlating packet evidence with iptables counters, and finally removing the rule to verify restoration.

The exercise demonstrates the importance of combining configuration evidence, application-level evidence, packet-level evidence and integrity hashes when performing a controlled network-forensic investigation.